

Projet de Master: Conception d'une plateforme citoyenne de signalement des incivilités

Année universitaire : 2025-2026

Master Cyber

Présenté par:

Yves Roland D. DEBO

Zakariae BOUAMAMA

Encadré par :

M. ZOUARI, M. YAACOUB et M. ELKETROUSSI

Table des matières

Introduction	2
1 Méthodologie et organisation du travail	3
1.1 Méthodologie de projet et cycle de vie	3
1.2 Gestion du code et outils collaboratifs	3
1.3 Répartition des Rôles et Partage de la Charge	4
1.4 Planning Réel de Réalisation	4
2 Conception détaillée de la solution	5
2.1 Modélisation des données	5
2.1.1 Justification du Modèle Document et Dénormalisation	6
2.2 Architecture fonctionnelle	7
2.2.1 Cinématique des États et Règles Métier	7
2.3 Architecture Logicielle	8
2.3.1 Modèle C4 (Niveau 2 : Containers)	8
2.3.2 Architecture Hexagonale	8
2.3.3 Convergence avec le Pattern CQRS Physique	9
3 Réalisation et Mise en Œuvre Technique	10
3.1 Développement Backend	10
3.1.1 Arborescence et structure du code source	10
3.2 Développement Frontend et Interfaces Utilisateurs	10
3.2.1 Application Mobile	10
3.2.2 Application Web	11
3.3 Infrastructure	11
3.3.1 Containerisation Isolée et Stratégie Réseau	11
3.3.2 Automatisation CI/CD (GitHub Actions & Ansible)	11
4 Stratégie de Tests	13
4.0.1 Méthodologie des Campagnes de Tests	13
4.0.2 Avantages de la Démarche pour le Projet	13
4.1 Matrice de Conformité et Indicateurs de Réussite	14
Conclusion	15
Annexes	16

Introduction

La croissance démographique et l'urbanisation rapide des métropoles africaines, particulièrement en Côte d'Ivoire, s'accompagnent de défis majeurs en matière de gestion du cadre de vie. Les municipalités font face à une multiplication des dysfonctionnements urbains et des incivilités : dépôts sauvages d'ordures, caniveaux obstrués, écoulements d'eaux usées ou encore défaillances de l'éclairage public. Bien que ces anomalies soient quotidiennement constatées par les résidents, leur signalement reste informel, fragmenté et difficilement traçable par les autorités compétentes.

Plusieurs initiatives occidentales de *civic tech*, telles que *FixMyStreet* ou *SeeClickFix*, ont démontré l'efficacité du numérique pour rapprocher les administrés de leur gestion locale. Cependant, ces solutions reposent sur des structures institutionnelles préexistantes et adoptent une logique descendante qui peine à s'adapter aux réalités sociales et administratives ivoiriennes. Face au manque de confiance ou au sentiment d'inefficacité qui freinent parfois l'implication citoyenne, il devient essentiel de réinventer les mécanismes de participation collective.

Le projet **Djahiri** est né de cette ambition : concevoir et déployer une plateforme hybride sécurisée de signalement des incivilités urbaines, pensée spécifiquement pour le contexte africain. L'originalité de Djahiri réside dans l'intégration d'un écosystème communautaire et ludique. En associant une démarche éducative à un système de gamification, la plateforme valorise chaque action individuelle et stimule un engagement civique durable.

Ce rapport présente l'ensemble de la démarche d'ingénierie logicielle menée pour concrétiser cette vision, depuis la formalisation des besoins et la conception architecturale jusqu'à la mise en œuvre technique, les pipelines de déploiement continu et la stratégie de validation par les tests.

1. Méthodologie et organisation du travail

La concrétisation d'un projet informatique d'envergure nécessite un cadre méthodologique rigoureux et une répartition claire des responsabilités au sein de l'équipe d'ingénierie. Ce chapitre détaille l'organisation de notre binôme et la démarche de gestion de projet adoptée.

1.1 Méthodologie de projet et cycle de vie

Pour répondre aux contraintes académiques et aux évolutions fonctionnelles du projet, nous avons appliqué une approche itérative inspirée des **méthodes agiles**. Le développement a été découpé en sprints de deux semaines, chacun se focalisant sur des objectifs intermédiaires précis et des livrables fonctionnels validés progressivement.

Ce cycle itératif nous a permis de maintenir une grande réactivité, notamment lors de l'intégration tardive d'une plateforme Web d'administration et de consultation citoyenne en ReactJS, venant compléter l'application mobile Flutter initialement planifiée.

1.2 Gestion du code et outils collaboratifs

La collaboration et le suivi du code source ont été entièrement centralisés au sein d'une organisation dédiée sur la plateforme GitHub.

- **Stratégie de Branching** : Le versioning repose sur une structure stricte à trois branches principales stables (`main` pour la production, `staging` pour la pré-production, et `dev` pour l'intégration). Chaque nouvelle fonctionnalité fait l'objet d'une branche isolée (ex : `feat/auth`).
- **Revue de Code** : La fusion d'une branche de fonctionnalité vers la branche `dev` requiert obligatoirement l'ouverture d'une *Pull Request* (PR) et une validation croisée entre les deux membres du binôme.
- **Qualité et Commits** : Afin de standardiser l'historique des modifications, nous appliquons la convention des *Conventional Commits* (ex : `feat: add JWT validation`).

1.3 Répartition des Rôles et Partage de la Charge

Bien que l'évaluation finale du projet soit individualisée, le travail a été mené de manière collaborative en séparant nos expertises clés pour maximiser l'efficacité du binôme :

- **Yves Roland DEBO** : Responsable de la conception et du développement de l'application mobile Flutter, de la modélisation de la base de données MongoDB Atlas, de l'architecture hexagonale du backend custom ainsi que de la configuration de l'infrastructure Cloud et réseau (VPS, Docker, Nginx, pipelines CI/CD via GitHub Actions et Ansible).
- **Zakariae BOUAMAMA** : Responsable de l'implémentation de l'API REST Spring Boot, du développement des modules métiers, de la réalisation du frontend Web et du panel d'administration sous ReactJS, ainsi que de la rédaction de la couverture globale de tests unitaires et d'intégration.

1.4 Planning Réel de Réalisation

Le projet s'est déroulé sur une période stricte de 12 semaines. Le tableau 1 synthétise le calendrier réel des livrables et des phases d'ingénierie logicielle respectées par notre équipe.

2. Conception détaillée de la solution

La réussite de la plateforme **Djahiri** repose sur une phase de modélisation rigoureuse. Ce chapitre détaille la structure de la persistance des données, les cinématiques fonctionnelles de modération ainsi que les choix architecturaux structurant le code source du backend.

2.1 Modélisation des données

Bien que **MongoDB** soit un système de gestion de base de données NoSQL orienté documents et par nature sans schéma, nous avons appliqué des règles strictes de typage et de structuration. Ce choix technique offre la flexibilité nécessaire à la manipulation de données géospatiales et multimédias tout en garantissant l'intégrité référentielle par l'usage d'identifiants uniques universels pour l'ensemble des relations inter-collections.

En ce qui concerne l'architecture de persistance, elle est segmentée en trois collections principales, optimisées pour minimiser les opérations d'entrée/sortie et éliminer le recours à des jointures applicatives complexes.

*Collection **users***

Cette collection centralise les informations de profil, les configurations de sécurité et l'état de progression lié à la gamification :

- **Identification et Métadonnées** : `_id` (UUID, clé primaire), `name` (Chaîne), `email` (Chaîne, unique), `phone` (Chaîne), `createdAt` (Date), `updatedAt` (Date).
- **Sécurité** : `passwordHash` (Chaîne), `authProvider` (Énumération : PASSWORD, GOOGLE), `role` (Énumération : CITIZEN, ADMIN), `suspended` (Booléen).
- **Gamification embarquée** : `points` (Entier), `badges` (Tableau de sous-documents embarqués contenant `badgeId`, `obtainedAt` et `activated`).

Collection *reports*

Collection centrale et la plus volumineuse du système, elle encapsule l'intégralité du cycle de vie d'une incivilité :

- **Localisation spatio-temporelle** : `_id` (UUID), `latitude` (Double), `longitude` (Double), `city` (Chaîne), `municipality` (Chaîne), `createdAt` (Date).
- **Contenu du signalement** : `photoUrl` (Chaîne, URL Cloudinary), `description` (Chaîne), `category` (Énumération : ORDURES, VOIRIE, PROPLETE, AUTRE).
- **Suivi et Modération** : `status` (Énumération : EN ATTENTE, VALIDE, REJETE, RESOLU), `authorId` (UUID, référence vers `users`), `adminComment` (Chaîne).
- **Interactions sociales embarquées** : `reactions` (Tableau d'UUID de `users` ayant soutenu le signalement), `comments` (Tableau de documents complets embarqués contenant `commentId`, `userId`, `userName`, `text`, `createdAt`).

Collection *badges*

Cette collection fait office de référentiel statique pour le moteur de règles de gamification :

- **Attributs** : `_id` (UUID), `label` (Chaîne, ex : "Éco-Citoyen"), `description` (Chaîne), `threshold` (Entier, nombre de validations requis pour l'obtention), `iconUrl` (Chaîne).

2.1.1 Justification du Modèle Document et Dénormalisation

Pour répondre aux fortes contraintes de charge d'une application mobile, nous avons privilégié la dénormalisation via les **documents embarqués** (*Embedded Documents*) pour les structures à cardinalité contrôlée, telles que les commentaires d'un signalement ou les badges d'un utilisateur.

Dans un modèle relationnel classique, l'affichage du fil d'actualité nécessiterait des jointures coûteuses entre les tables Signalements, Commentaires et Utilisateurs. Sous MongoDB Atlas, la récupération d'un document report s'effectue en une seule opération d'I/O (*Single-Document Atomicity*), optimisant drastiquement les temps de réponse de l'API et l'expérience utilisateur finale.

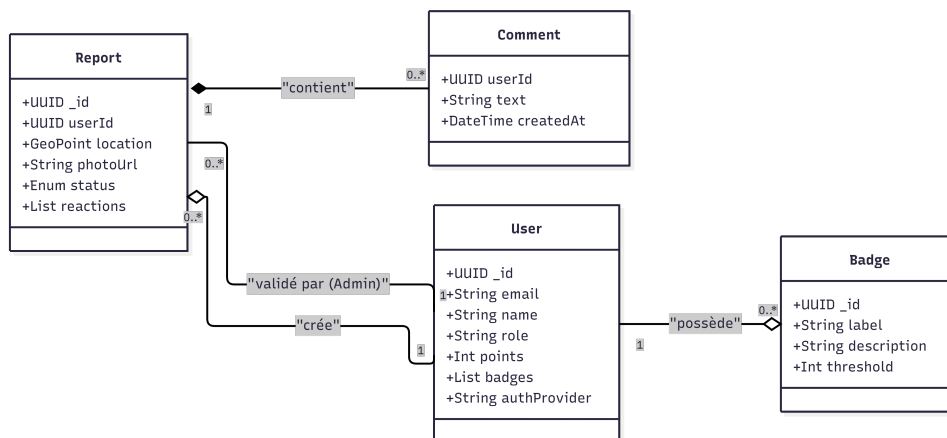


FIGURE 2.1 – Modélisation logique de la base de données dénormalisée MongoDB

2.2 Architecture fonctionnelle

Le cœur fonctionnel de la plateforme repose sur la transition d'états d'un signalement, régie par des règles métier strictes assurant la confiance au sein de la communauté.

2.2.1 Cinématique des États et Règles Métier

Un signalement traverse obligatoirement quatre états successifs :

1. **EN_ATTENTE** : Statut initial dès la soumission par le citoyen. La photo et le GPS sont figés. L'appareil photo de l'application mobile est contraint à une capture en temps réel pour empêcher l'import de médias falsifiés depuis la galerie.
2. **VALIDE / REJETE** : Un modérateur (compte ADMIN) examine l'enquête depuis le panel d'administration Web. Deux issues sont possibles :
 - *Rejet* : Le signalement est archivé visuellement, un `adminComment` explicatif est obligatoire pour notifier et éduquer l'utilisateur.
 - *Validation* : Le signalement devient public sur le fil d'actualité. Cette action déclenche immédiatement le moteur de règles asynchrone : attribution automatique de 10 points à l'auteur, vérification des seuils pour l'octroi d'un nouveau badge, et publication d'un événement métier.
3. **RESOLU** : Marque la fin de l'incident après intervention des services municipaux ou de la communauté.

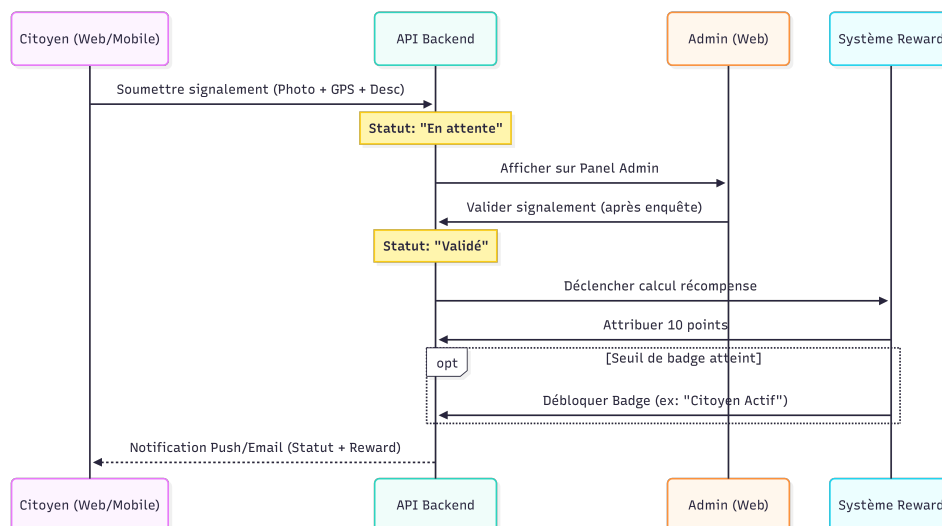


FIGURE 2.2 – Diagramme de séquence et cinématique d'états du processus de modération

2.3 Architecture Logicielle

La pérennité de l'application **Djahiri** est garantie par l'adoption de patterns architecturaux découplant rigoureusement la logique métier des frameworks techniques.

2.3.1 Modèle C4 (Niveau 2 : Containers)

Pour expliciter l'organisation macroscopique du système, le diagramme de conteneurs (Fig. 2.3) illustre la répartition des responsabilités logicielles. L'application Flutter destinée aux citoyens et l'application Web ReactJS destinée et aux citoyens et aux administrateurs n'attaquent jamais directement la persistance. Ils consomment de manière sécurisée l'API REST unifiée du backend Spring Boot, qui orchestre à son tour les services tiers : Firebase pour les *push notifications*, Brevo pour l'envoi des notifications e-mail et Cloudinary pour le stockage des fichiers multimédias.

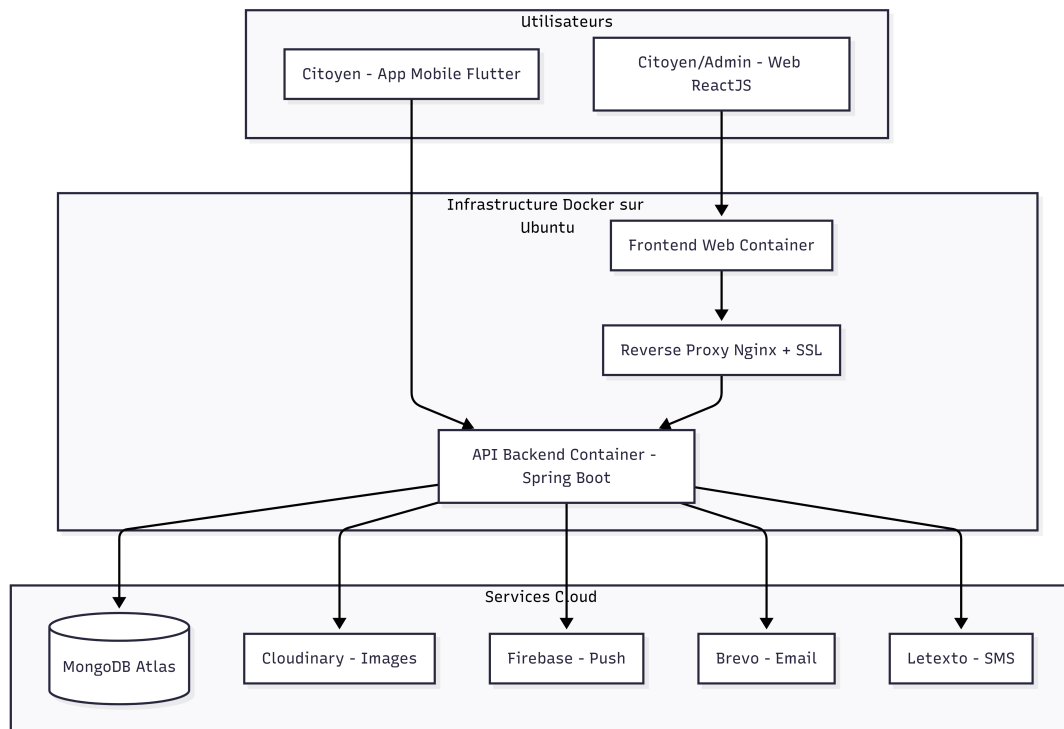


FIGURE 2.3 – Diagramme de conteneurs C4 (Niveau 2) de la solution globale

2.3.2 Architecture Hexagonale

Au sein du conteneur backend, nous avons implémenté une **Architecture Hexagonale** (*Ports & Adapters*) afin d'appliquer le principe d'inversion des dépendances. La structure physique des répertoires isole totalement le métier au centre de l'application :

- **Couche Domaine** (`domain/`) : C'est le centre de l'hexagone, totalement agnostique des bibliothèques externes. Elle contient les règles pures, les entités métier (`context/`), les structures d'échange internes (`request/`) et les événements asynchrones (`events/`).

- **Couche Présentation** (`rest/`) : Elle intercepte les requêtes HTTP, valide les payloads via les `dto/` entrants, effectue le mapping vers les objets du domaine et délègue l'exécution aux cas d'utilisation.
- **Couche Persistance** (`database/`) : Agit comme un adaptateur. Elle traduit les ordres du domaine en requêtes de base de données en manipulant des `documents/` MongoDB spécifiques, évitant ainsi de polluer le domaine avec des problématiques de stockage.
- **Couches Infrastructure et Sécurité** (`infrastructure/`, `security/`) : Isolent les configurations transverses telles que les filtres d'interception des jetons JWT ou les clients HTTP de communication avec les services externes (Firebase, Brevo, Cloudfire).

2.3.3 Convergence avec le Pattern CQRS Physique

Afin d'éviter d'asphyxier le domaine par des modèles de données trop complexes, nous avons fusionné l'architecture hexagonale avec une implémentation physique du pattern **CQRS** (*Command Query Responsibility Segregation*) à l'intérieur même de l'arborescence à travers les packages `in` et `out` (Tab. 2).

Grâce à cette segmentation, les requêtes d'affichage du fil d'actualité (`out`) exploitent des projections ciblées de MongoDB pour ne retourner que les sous-ensembles de données nécessaires au composant ReactJS ou Flutter. À l'inverse, les processus d'écriture (`in`)instancient l'entité de domaine complète pour exécuter les validations d'intégrité algorithmiques (Fig. 1).

3. Réalisation et Mise en Œuvre Technique

Ce chapitre détaille la phase de réalisation de la plateforme **Djahiri**. Il met en lumière l'implémentation concrète des choix technologiques, la structure du code source et l'environnement de déploiement automatisé garantissant la stabilité de la solution.

3.1 Développement Backend

Le serveur central de Djahiri est propulsé par le framework **Spring Boot (Java 21)**. L'intégralité du développement tente de respecter les frontières étanches imposées par l'architecture hexagonale validée en phase de conception.

3.1.1 Arborescence et structure du code source

Pour assurer la maintenabilité au sein de l'équipe, le projet adopte une arborescence stricte où le domaine métier est placé au centre, sans aucune pollution technique :

- `common/` : Regroupe les structures transverses, notamment la classe `ValidationControllerAdvice` étendant `ResponseEntityExceptionHandler` pour centraliser la capture et le formatage des exceptions HTTP.
- `database/` : Contient les connecteurs MongoDB, les requêtes de modification, de création et de lecture des documents.
- `domain/` : Renferme les `usecases/` algorithmiques, les définitions de `ports/`, les énumérations métiers (`ReportStatus`, `AuthProvider`) et les constantes d'attribution des récompenses (`VALIDATED_REPORT_REWARD_POINTS = 10`).
- `rest/` : Héberge la couche d'exposition comprenant les contrôleurs d'API documentés via des annotations `OpenAPI/Swagger`.

3.2 Développement Frontend et Interfaces Utilisateurs

L'architecture applicative est scindée en deux clients distincts pour répondre aux besoins spécifiques des utilisateurs.

3.2.1 Application Mobile

Développée avec le framework **Flutter**, l'application mobile cible les terminaux Android et iOS à partir d'une base de code unique. Elle adopte une *feature-based architecture* segmentant l'application par modules fonctionnels (Authentification, Fil d'actualité, Signalement, Profil).

L'accès natif aux capteurs de l'appareil permet la capture instantanée de la caméra pour figer le cliché et l'interrogation du composant GPS pour instancier les variables latitude et longitude indispensables au géocodage inverse opéré par l'API.

3.2.2 Application Web

Développé sous **ReactJS**, le portail Web offre deux niveaux de lecture grâce à une gestion fine du contrôle d'accès basé sur les rôles (*RBAC*) :

- **Vue Citoyenne** (*role = CITIZEN*) : Donne une alternative aux utilisateurs n'ayant pas téléchargé l'application mobile. Toutes les fonctionnalités présentes sur l'application mobile sont également présentes sur l'application web.
- **Vue Administrateur** (*role = ADMIN*) : Donne accès au tableau de bord décisionnel regroupant les indicateurs clés (Fig. ??). C'est depuis cette interface (Fig. ??) que les modérateurs saisissent leurs retours d'enquête, appliquent le statut de validation et déclenchent l'octroi automatique des points et des badges en base de données.

3.3 Infrastructure

Pour s'affranchir des problématiques de configuration d'environnement lors de la mise en production, l'intégralité de l'infrastructure a été virtualisée et automatisée.

3.3.1 Containerisation Isolée et Stratégie Réseau

Sur nos serveurs virtuels VPS Ubuntu, chaque composant s'exécute de manière hermétique à l'intérieur d'un conteneur **Docker** dédié. L'isolation est orchestrée par des fichiers *Dockerfile*, mappant de façon étanche les ports applicatifs internes sur l'hôte.

Un serveur de proxy inverse **Nginx**, installé sur la machine hôte du VPS, intercepte l'ensemble des requêtes entrantes sur le port 443. Il applique le chiffrement SSL, injecte les politiques de sécurité et route de manière transparente le trafic réseau vers le port 3000 (Frontend ReactJS) ou le port 8081 (API REST Spring Boot).

3.3.2 Automatisation CI/CD (GitHub Actions & Ansible)

Le cycle de déploiement est entièrement automatisé de bout en bout sans aucune intervention manuelle, éliminant les risques d'erreur humaine. Le flux respecte une stricte promotion du code à travers nos branches Git (Fig. 2) :

1. **Validation sur** *feat/** : Tout commit ou ouverture de Pull Request vers la branche *dev* déclenche un workflow GitHub Actions de compilation (*Build Only*) visant à valider l'intégrité syntaxique du code source.
2. **Déploiement en Staging** (*staging*) : La fusion du code sur la branche *staging* compile une image Docker dédiée, la pousse sur le *GitHub Container Registry* (GHCR) et lance un playbook **Ansible**. Ce dernier se connecte par SSH au

VPS de pré-production (82.165.xxx.xxx), télécharge l'image fraîchement taguée :staging et redémarre le conteneur de façon transparente.

3. **Mise en Production (main)** : Une fois les tests de recette validés en pré-production, une Pull Request vers la branche `main` réitère ce processus pour mettre à jour de manière identique le VPS de production (217.154.xxx.xxx) avec le tag :latest.

4. Stratégie de Tests

La phase de validation et de vérification constitue une étape cruciale de l'ingénierie logicielle, garantissant la conformité de la solution développée par rapport aux exigences du cahier des charges. Pour la plateforme **Djahiri**, notre démarche s'est principalement articulée autour de la validation du système par les utilisateurs finaux. En raison des contraintes de temps inhérentes au calendrier académique du projet, le développement initialement prévu de tests automatisés d'interface (tests unitaires graphiques et tests de composants) sur l'application mobile Flutter et le panel web ReactJS n'a pas pu être mené à son terme.

Pour pallier cette limite sans pour autant compromettre la qualité et la robustesse du livrable final, nous avons réorienté notre stratégie vers une méthodologie de **Tests d'Acceptation Utilisateur (UAT)** en conditions réelles, plaçant ainsi l'humain au centre du processus de validation.

4.0.1 Méthodologie des Campagnes de Tests

Les tests utilisateurs ont été menés de manière itérative auprès d'un panel représentatif de citoyens et de profils administrateurs, s'appuyant sur deux piliers :

- **Scénarios guidés** : Chaque testeur s'est vu confier une liste de tâches métiers précises à exécuter, telles que la création d'un signalement de nid-de-poule avec activation du composant GPS natif, ou le rejet motivé d'une anomalie depuis le panel de modération.
- **Collecte des retours et cahier de recettes** : Les anomalies visuelles, les latences réseau ou les éventuelles frustrations d'usage ont été rigoureusement consignées. Ces retours qualitatifs ont directement alimenté nos cycles de développement pour orienter les corrections d'anomalies lors des derniers Sprints.

4.0.2 Avantages de la Démarche pour le Projet

Bien que moins formelle qu'une suite de tests automatisés, cette approche par tests utilisateurs s'est révélée particulièrement vertueuse pour notre projet :

1. **Validation de l'UX/UI** : Elle a permis de valider l'ergonomie et l'intuitivité des interfaces à la volée, un aspect critique pour une application citoyenne destinée à être prise en main par un large public.
2. **Conformité de bout en bout (End-to-End)** : En réalisant des actions réelles — depuis la capture photo sur smartphone jusqu'à l'affichage de l'anomalie sur le panel web d'administration après transit par l'API REST —, les sessions de tests

ont validé l'intégration globale et la bonne communication inter-systèmes de la solution.

4.1 Matrice de Conformité et Indicateurs de Réussite

Pour conclure cette phase de vérification, le tableau 3 situé en annexe dresse le bilan final de conformité de la plateforme vis-à-vis des indicateurs de réussite fonctionnels et techniques définis en début de projet.

Conclusion

Le projet **Djahiri** est né d'une ambition forte : mettre les avancées de la *civic tech* au service de l'amélioration du cadre de vie et de la gestion des incivilités urbaines en Côte d'Ivoire. À l'issue de ces douze semaines d'ingénierie intensive, ce projet nous a permis de traverser l'intégralité des étapes du cycle de vie d'un logiciel complexe, depuis la formalisation du besoin jusqu'à la mise en place d'une infrastructure cloud automatisée. Sur le plan technique, les objectifs principaux du cœur de la solution ont été atteints avec succès. Nous avons pu construire un backend robuste et découplé, deux interfaces clientes fonctionnelles, ainsi que d'une infrastructure DevOps industrielle. La mise en œuvre de tests d'acceptation utilisateur (UAT) s'est avérée déterminante pour valider la communication inter-systèmes et la conformité des flux de bout en bout, répondant ainsi pleinement aux exigences de qualité fixées à l'entame du développement.

Toutefois, malgré un rythme de développement soutenu en sprints agiles, les contraintes temporelles inhérentes au calendrier académique nous ont imposées des choix de priorisation rigoureux. Certains modules initialement envisagés n'ont pu être menés à leur terme et constituent les limites actuelles du système, notamment l'automatisation des tests unitaires d'interface sur les clients graphiques, la gestion avancée des profils utilisateurs, ainsi que le couplage du moteur de gamification à un catalogue réel de récompenses partenaires. Ces restrictions dessinent néanmoins une feuille de route claire pour les évolutions futures du projet, qui s'orienteront naturellement vers la finalisation de ce module économique B2B2C et l'intégration de l'intelligence artificielle pour la pré-classification automatisée des images de signalements.

En conclusion, bien que perfectible, la plateforme Djahiri pose des fondations d'ingénierie logicielle saines, modulaires et prêtes pour l'échelle. Ce projet nous a non seulement permis de consolider nos compétences de développeurs dans des environnements technologiques modernes et exigeants, mais il nous a également sensibilisés aux critères de rigueur, d'autonomie et de compromis technique indispensables à la réussite de futurs projets industriels de grande envergure.

Annexes

Cette section rassemble les documents techniques, diagrammes d'architecture, matrices de suivi et plannings de réalisation qui soutiennent les analyses et les choix d'ingénierie logicielle détaillés tout au long de ce rapport.

Le code source du projet est disponible sur GitHub : https://github.com/djahiri-app/projet_master

Phase / Livrable	Période / Date Limite	Objectifs et Productions Réalisés
Livrable 1 (L1)	31 Mars 2026	Cadrage, objectifs, planning et méthodologie.
Phase 1 : Conception	Semaines 1 à 4	Modélisation DB, diagrammes C4, architecture hexagonale.
Phase 2 : Backend & Web	Semaines 3 à 8	API Spring Boot, sécurité JWT, Panel Web ReactJS.
Phase 3 : App Mobile	Semaines 7 à 10	Application Flutter, géolocalisation, intégration API.
Livrable 2 (L2)	28 Avril 2026	Rapport d'avancement technique et conception finalisée.
Phase 4 : Validation & CI/CD	Semaines 10 à 11	Stratégie de tests unitaires/E2E, déploiements automatisés VPS.
Phase 5 : Clôture	Semaine 12	Finalisation du dépôt GitHub, documentation et rédaction.

TABLE 1 – Planning chronologique réel des phases du projet Djahiri

Flux Logiciel	Rôle et Implémentation Métier	Composants Applicatifs Impactés
in (Commandes)	Capture l'intention de modification de l'état du système. Strictement aligné sur l'écriture.	rest/dto/in, domain/ports/in (Use Cases), adapters/in.
out (Requêtes)	Optimisé exclusivement pour l'extraction rapide et l'affichage des données.	rest/dto/out, domain/ports/out, database/adapters/out.

TABLE 2 – Alignement physique du pattern CQRS sur les frontières de l'architecture hexagonale

Objectif Initial	Mécanisme de Validation	Statut Final
Sécurisation de l'API REST	Tests d'intégration + Filtres JWT	Validé
Persistance et intégrité	Schémas et règles de Use Cases	Validé
Figurer l'authenticité des médias	Contrainte de capture caméra native (Mobile & Web)	Validé
Communication inter-systèmes	Flux complets validés lors des sessions UAT	Validé
Automatisation des livraisons	Déploiements idempotents validés en Staging/Prod	Validé

TABLE 3 – Matrice de conformité et état d'avancement des indicateurs de réussite

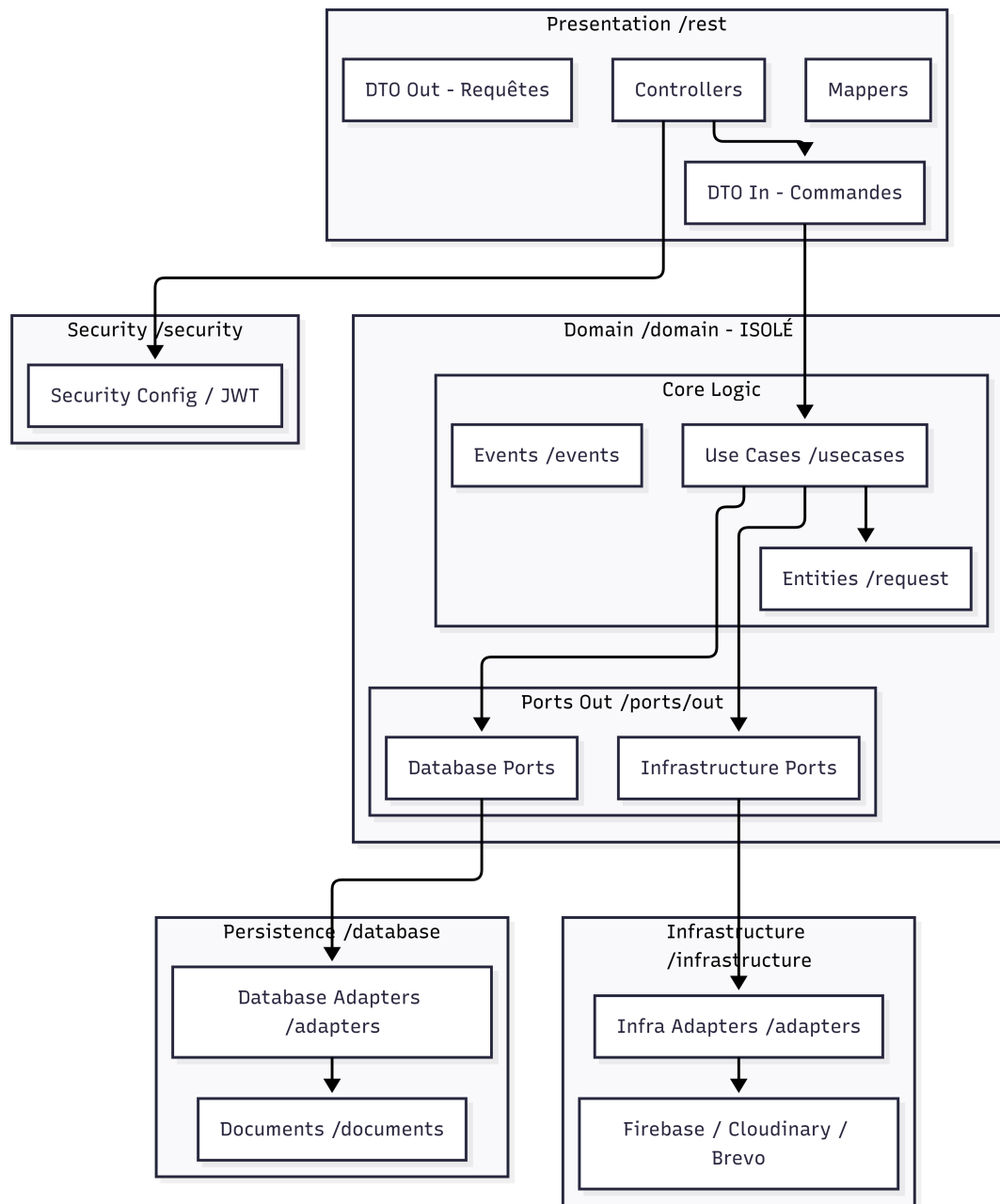


FIGURE 1 – Architecture hexagonale et routage des flux CQRS du backend Djahiri

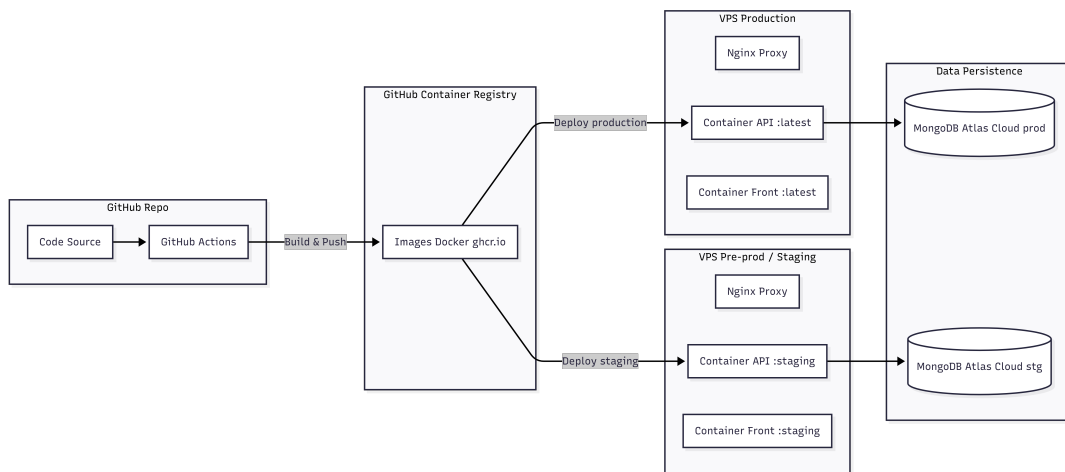


FIGURE 2 – Stratégie de déploiement et promotion du code source